

Boosting & SVM

Motivation for Boosting

Question: Could we address the shortcomings of single decision trees models in some other way?

For example, rather than performing variance reduction on complex trees, can we decrease the bias of simple trees - make them more expressive?

Can we learn from our mistakes?

A solution to this problem, making an expressive model from simple trees, is another class of ensemble methods called ***boosting***.

Boosting

NEW IDEA

- **Boosting** methods are general algorithms which combine several "**weak learners**" to produce a strong rule.
- The first implementation of Boosting was '**Adaboost**' invented by Robert Schapire and Yoav Freund in 1996.
- Boosting algorithms are **fast**, easy to compute and very accurate and are the de-facto optimization tree algorithms.

Get Good understanding of Boosting Concept to Get Grade in CS-351

OPTION #1

1. Steal the time-stone from Dr. Strange.

2. Go back to 1996 and meet Rob Schapire and Yoav Freund.
3. Follow their work for at least a decade to understand everything about boosting.
4. Return to the present and nail the test.
5. Repeat for another test

OPTION #2

STEP #1:

Go to the library and get previous year question papers.



STEP #2:

Find a helpful student and ask her to give you a "rule of thumb" to get at least some answers right.



DOES THE "RULE" WORK ?

Test out the rule.
It worked 60% of the time. Not bad!!

STEP #3:

Find a TA and ask him to also give you a "rule of thumb" to get at least some answers right.



DOES THE "RULE" WORK ?

Test out the new rule.
It works well on difficult problems! But a few problems persist.



STEP #4:

Call your favorite professor and focus on the ones you got wrong before!

make sure to focus on the ones you got wrong before.

OPTION #2

DOES THE "RULE" WORK
?

Test out the new rule.

The new rule works well
on the difficult
problems!



STEP #5:

Combine the rules, but
pay more **attention** to
the ones that were more
often right

Accuracy: 60%



Accuracy: 64%



Accuracy: 70%



$Strategy = \alpha * Rule_1 +$

$\beta * Rule_2 +$

$\gamma * Rule_3$

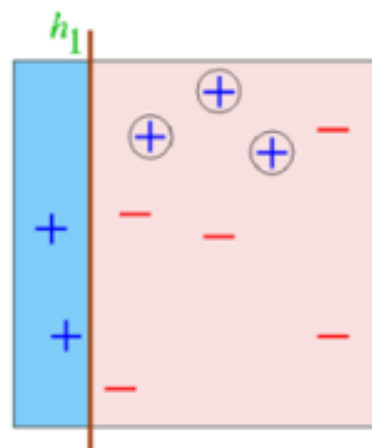
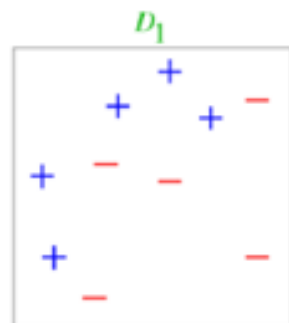
FINAL STEP:

Take the test with these
approximate rules,
weighted by how well
each rule performed.

A+

Boosting

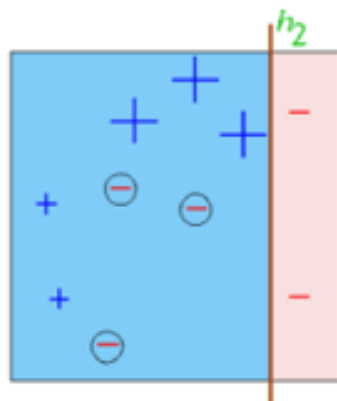
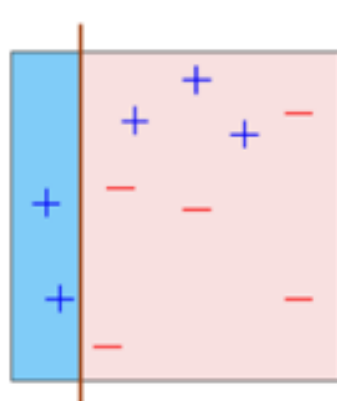
First classifier



$$\epsilon_1 = 0.30$$

$$\alpha_1 = 0.42$$

First 2 classifiers

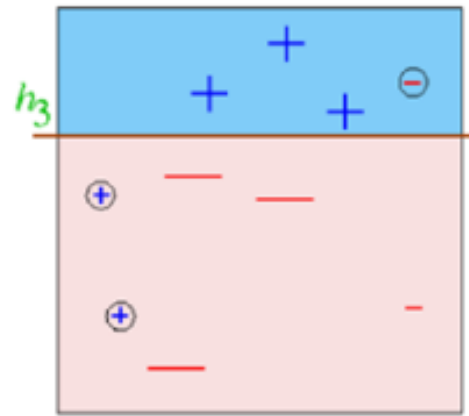
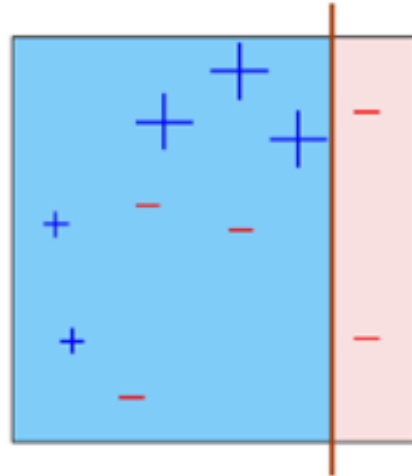
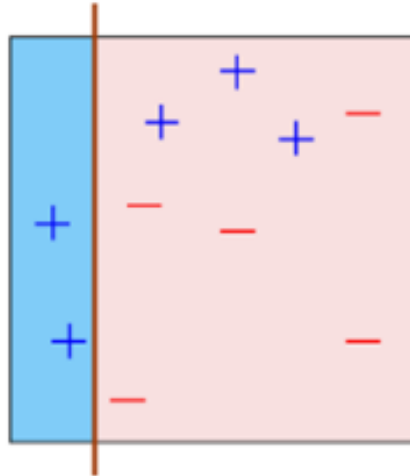


$$\epsilon_2 = 0.21$$

$$\alpha_2 = 0.65$$

Adaboost combines multiple weak classifiers into a strong classifier by weighting their outputs. The final decision is a weighted sum of these weak classifiers.

Boosting



$$\epsilon_3 = 0.14$$

$$\alpha_3 = 0.92$$

$$H_{\text{final}} = \text{sign} \left(0.42 \begin{array}{|c|} \hline \text{blue} \\ \hline \end{array} + 0.65 \begin{array}{|c|} \hline \text{blue} \\ \hline \end{array} + 0.92 \begin{array}{|c|} \hline \text{blue} \\ \hline \end{array} \right) = \begin{array}{|c|} \hline \text{blue} \\ \hline \end{array}$$

The final diagram shows the result of the boosting process. It is a square plot with a vertical line at $x \approx 0.1$ and a horizontal line at $y \approx 0.4$. The regions are colored blue and red. There are 4 blue '+' symbols and 2 red '-' symbols. The blue region contains 3 '+' symbols and 1 '-' symbol, and the red region contains 1 '+' symbol and 1 '-' symbol.

Suppose we have training data $\{(x_i, y_i)\}_{i=1}^N$, $y_i \in \{1, -1\}$

- Initialize equal weights for all observations $w_i = 1/N$
- At each iteration t :
 1. **Train a stump** G_t using data weighted by w_i
 2. **Compute the misclassification error** adjusted by w_i
 3. **Compute the weight of the current tree** α_t
 4. **Reweight each observation** based on prediction accuracy

- Calculate the weighted misclassification error

$$err_t = \frac{\sum_{i=1}^N w_i I(y_i \neq G_t(x_i))}{\sum_{i=1}^N w_i}$$

- Use the error score to weight the current tree in the final classifier: determines how much influence this classifier has in the final decision.

$$\alpha_t = \log \left(\frac{1 - err_t}{err_t} \right)$$

A classifier with 50% accuracy is given a weight of zero;

- Use misclassification error and tree weight to reweight the training data:

$$w_i \leftarrow w_i \exp[\alpha_t I(y_i \neq G_t(x_i))]$$

- **Final prediction is a weighted sum of the predictions from each stump:**

$$G(x) = \text{sign} \left[\sum_{t=1}^T \alpha_t G_t(x) \right]$$

More accurate trees are weighted higher in the final model.

At test time, evaluate all classifiers and return the weighted sum.

- Use it as it is for regression
- Compare the sum across categories and let the category with the highest weighted sum win in case of classification

Put Together: AdaBoost

1. Start with equal weights
2. Train weak classifier
3. Compute weighted error
4. Compute classifier importance (α)
5. Increase weight of misclassified points
6. Train next classifier on updated weights
7. Repeat

Final prediction = **weighted majority vote** of all weak models.

AdaBoost Summary

- **Strengths:**

- Fast and simple to implement
- No hyperparameters (except for T)
- Very few assumptions on base models

- **Weaknesses:**

- Can be susceptible to noise/outliers when there is insufficient data
- No way to parallelize
- Small gains over complex base models
- **Specific to classification!**

Boosting as Gradient Descent

- Both algorithms: **new model** = **old model** + **update**

- Gradient Descent:**

$$\theta_{t+1} = \theta_t - \alpha \cdot \nabla_{\theta} L(\theta_t; Z)$$

- Boosting:**

$$F_{t+1}(x) = F_t(x) + \beta_{t+1} \cdot f_{t+1}(x)$$

- Here, $F_t(x) = \sum_{i=1}^t \beta_i \cdot f_i(x)$

Boosting as Gradient Descent

- Assuming $\beta_t = 1$ for all t , then:

$$F_t(x_i) + f_{t+1}(x_i) = F_{t+1}(x_i)$$

This is the new base model
we are trying to train at time t

Boosting as Gradient Descent

- Assuming $\beta_t = 1$ for all t , then:

$$F_t(x_i) + f_{t+1}(x_i) = F_{t+1}(x_i) \approx y_i$$

- Rewriting this equation, we have

$$f_{t+1}(x_i) = F_{t+1}(x_i) - F_t(x_i) \approx \underbrace{y_i - F_t(x_i)}$$

“residuals”, i.e., error of the current model

Boosting as Gradient Descent

- In other words, at each step, boosting is training the next model f_{t+1} to approximate the residual:

$$f_{t+1}(x_i) \approx \underbrace{y_i - F_t(x_i)}$$

“residuals”, i.e., error of the current model

Q: does this have any connection to residual networks?

- **Idea:** Train f_{t+1} directly to predict residuals $y_i - F_t(x_i)$
- **This strategy works for regression as well!**

Boosting as Gradient Descent

- **Algorithm:** For each $t \in \{1, \dots, T\}$:

- **Step 1:** Train f_{t+1} using dataset

$$Z_{t+1} = \{(x_i, y_i - F_t(x_i))\}_{i=1}^n$$

Q: Is this label in fact the gradient?

A: Yes!

- **Step 2:** Take

$$F_{t+1}(x) = F_t(x) + f_{t+1}(x)$$


- Return the final model F_T

For fastest improvement on MSE,
this should be along the direction of steepest descent

Boosting as Gradient Descent

Generalizing ...

- Consider losses of the form

$$L(F; Z) = \frac{1}{n} \sum_{i=1}^n \tilde{L}(F(x_i); y_i)$$

- In other words, sum of individual label-level losses $\tilde{L}(\hat{y}; y)$ of a prediction $\hat{y} = F(x)$ if the ground truth label is y
- For example, $\tilde{L}(\hat{y}; y) = \frac{1}{2} (\hat{y} - y)^2$ yields the MSE loss

Boosting as Gradient Descent

- Residuals are the gradient of the squared error $\tilde{L}(\hat{y}; y) = \frac{1}{2}(y - \hat{y})^2$:

$$-\frac{\partial \tilde{L}}{\partial \hat{y}}(F_t(x_i); y_i) = y_i - F_t(x_i) = \text{residual}_i$$

- For general $\tilde{L}$, instead of $\{(x_i, y_i - F_t(x_i))\}_{i=1}^n$ we can train f_{t+1} on

$$Z_{t+1} = \left\{ \left(x_i, -\frac{\partial \tilde{L}}{\partial \hat{y}}(F_t(x_i); y_i) \right) \right\}_{i=1}^n$$

Boosting as Gradient Descent

- **Algorithm:** For each $t \in \{1, \dots, T\}$:
 - **Step 1:** Train f_{t+1} using dataset

$$Z_{t+1} = \left\{ \left(x_i, -\frac{\partial \tilde{L}}{\partial \hat{y}}(F_t(x_i); y_i) \right) \right\}_{i=1}^n$$

- **Step 2:** Take

$$F_{t+1}(x) = F_t(x) + f_{t+1}(x)$$

- Return the final model F_T

Boosting as Gradient Descent

- Casts ensemble learning in the **loss minimization framework**
 - **Model family:** Sum of base models $F_T(x) = \sum_{t=1}^T f_t(x)$
 - **Loss:** Any differentiable loss expressed as

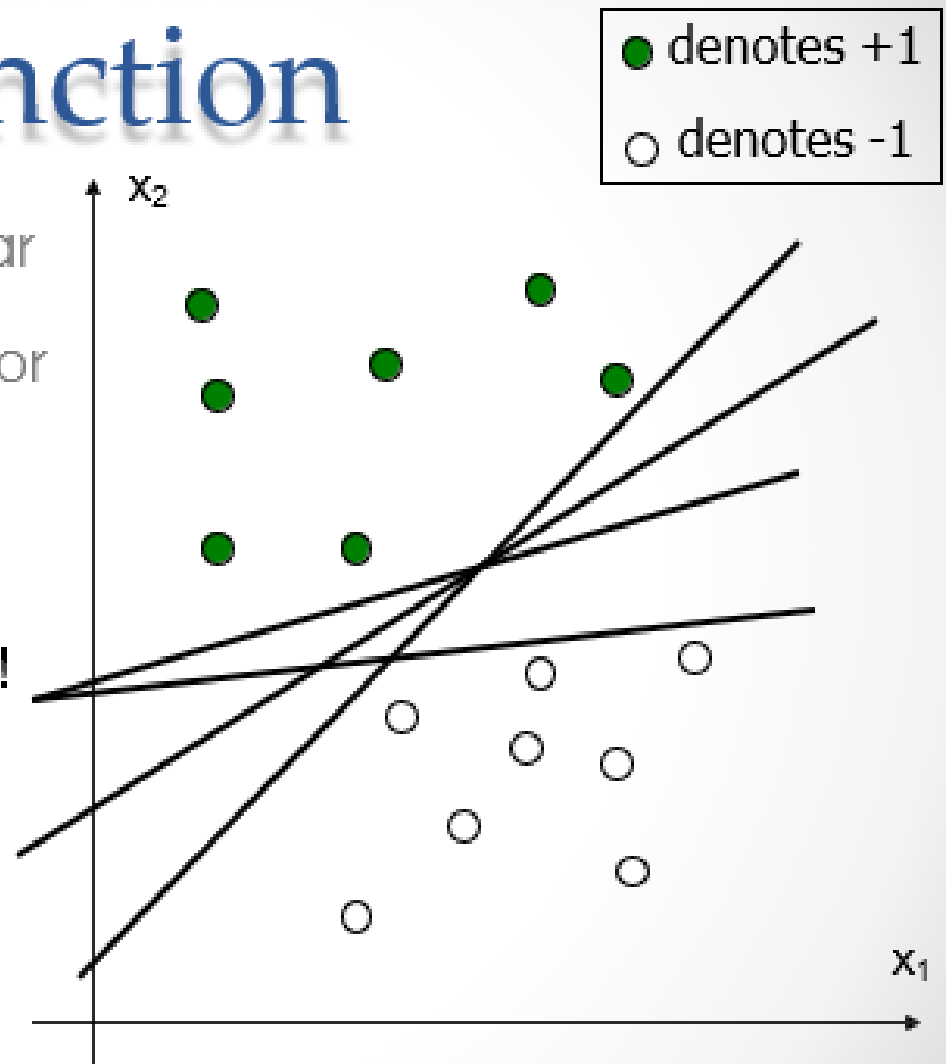
$$L(F; Z) = \sum_{i=1}^n \tilde{L}(F(x_i), y_i)$$

- Gradient boosting is a general paradigm for training ensembles with specialized losses (e.g., most NLL losses)

Support Vector Machines

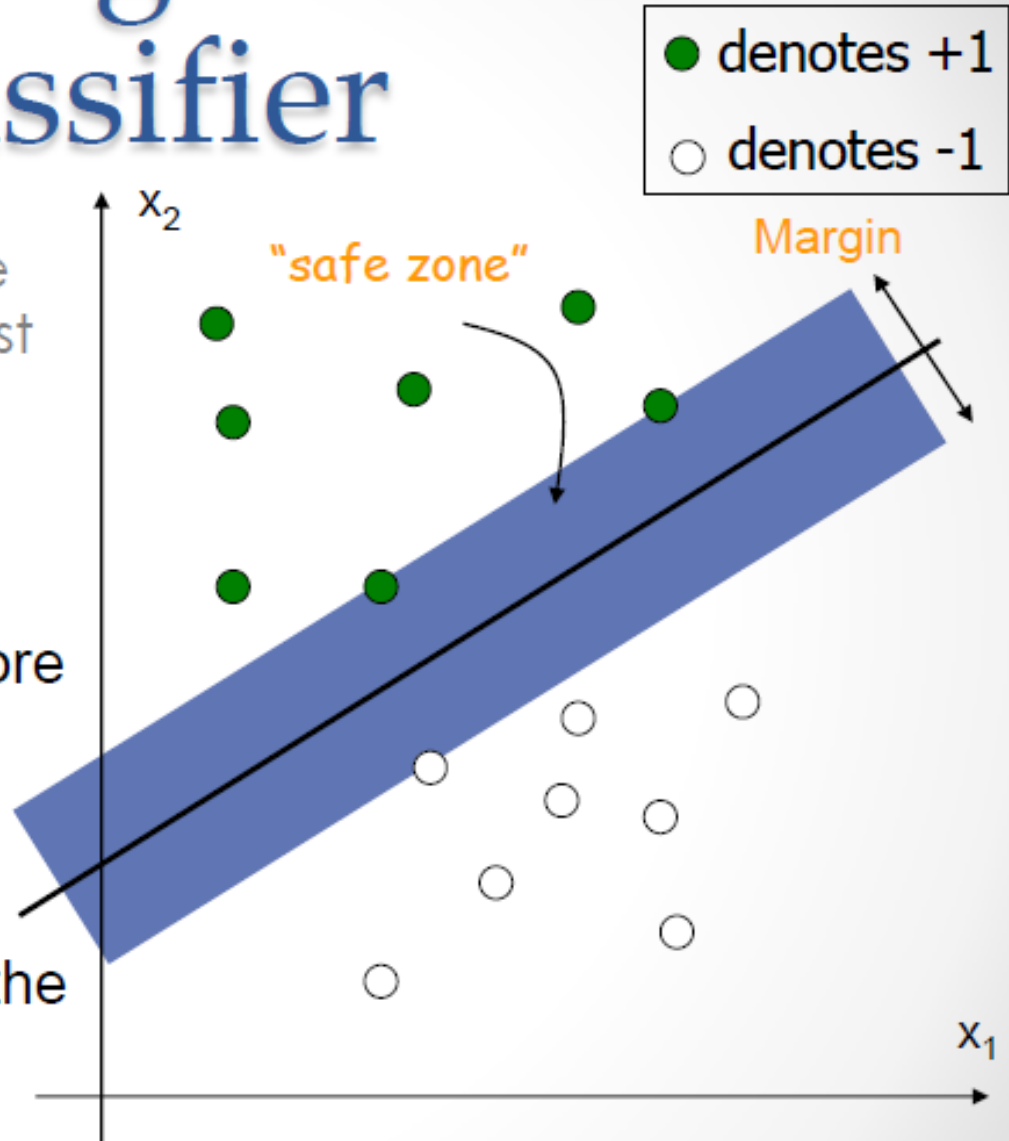
Linear Discriminant Function

- How would you classify these points using a linear discriminant function in order to minimize the error rate?
- Infinite number of answers!
- Which one is the best?



Large Margin Linear Classifier

- The linear discriminant function (classifier) with the maximum **margin** is the best
- Margin is defined as the width that the boundary could be increased by before hitting a data point
- Why it is the best?
 - The larger the margin the better generalization
 - Robust to outliers



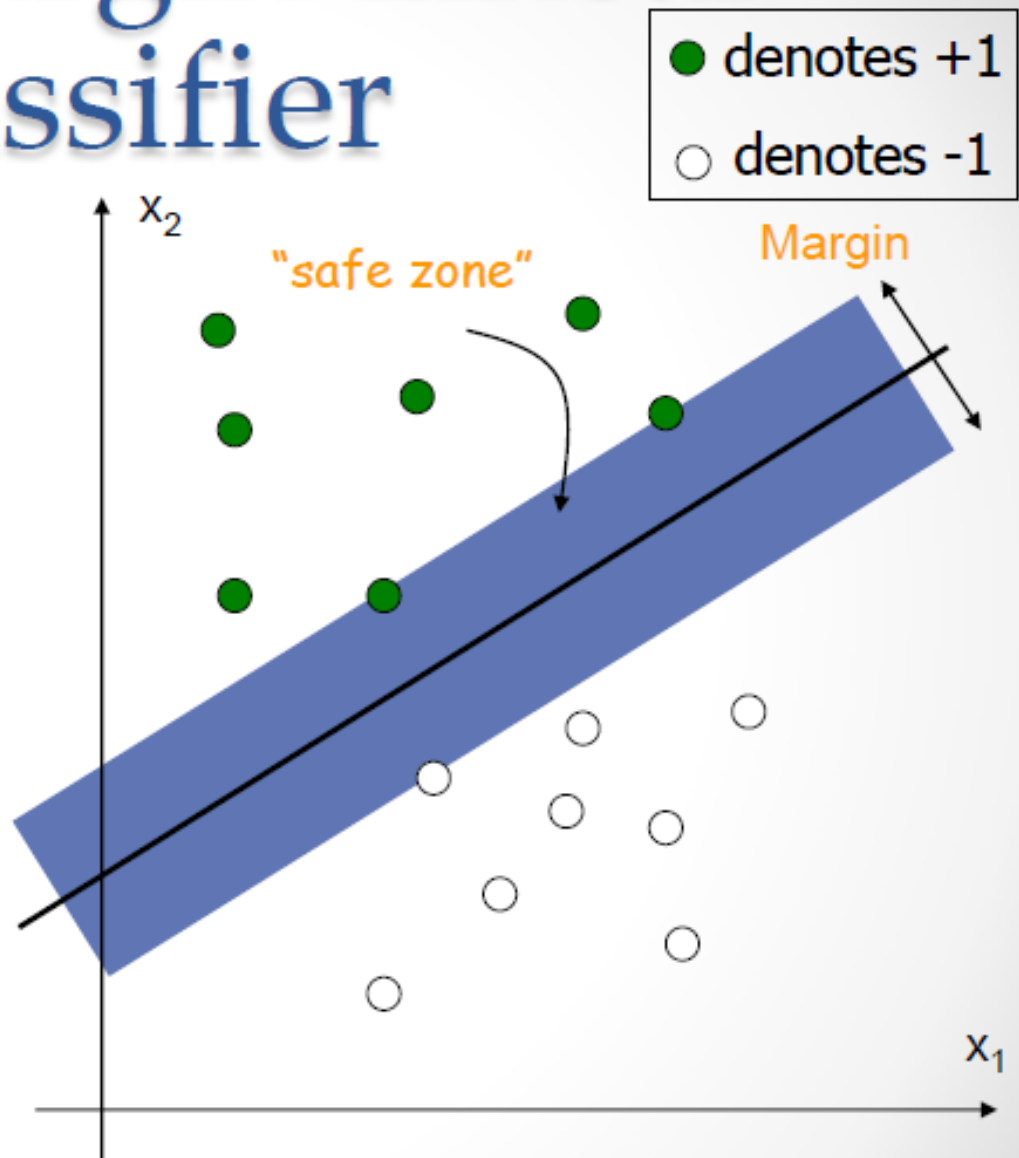
Large Margin Linear Classifier

- Aim: Learn a large margin classifier.
- Given a set of data points, define:

For $y_i = +1$, $\mathbf{w}^T \mathbf{x}_i + b \geq 1$

For $y_i = -1$, $\mathbf{w}^T \mathbf{x}_i + b \leq -1$

- Give an algebraic expression for the width of the margin.



Algebraic Expression for Width of a Margin

Given 2 parallel lines with equations

$$ax + by + c_1 = 0$$

and

$$ax + by + c_2 = 0$$

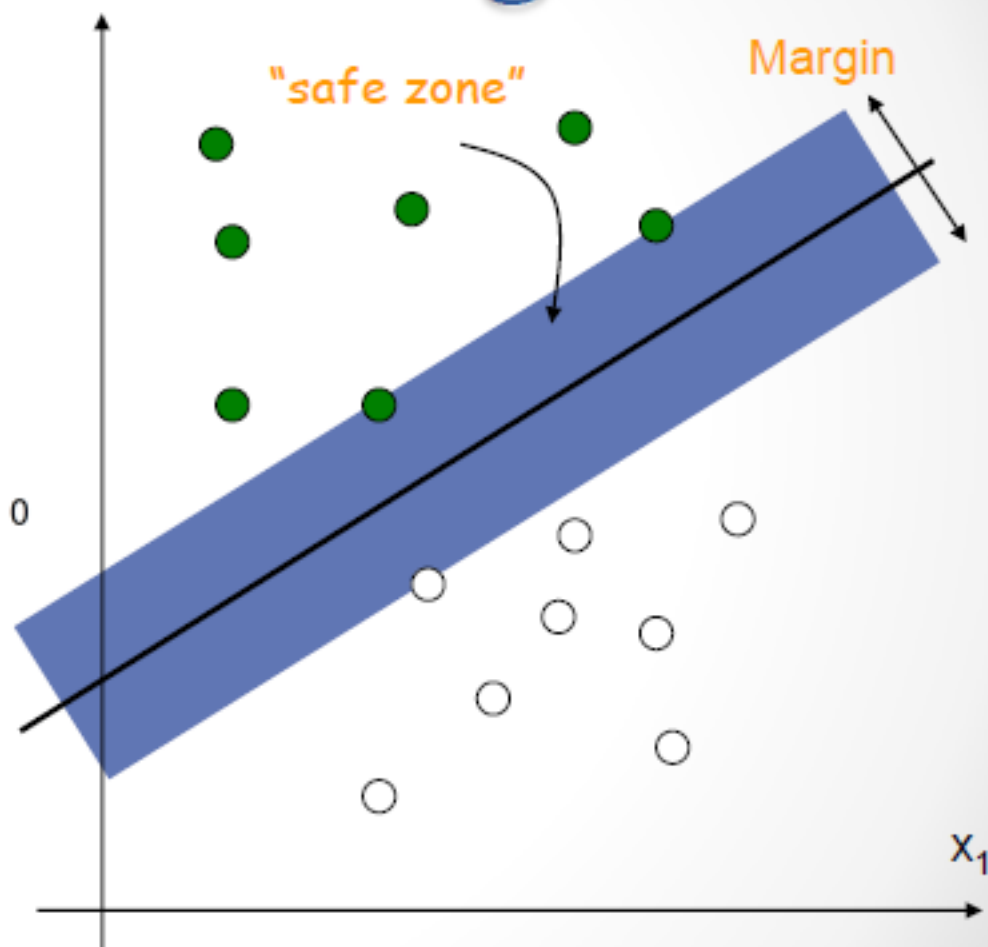
the distance between them is given by:

$$d = \frac{|c_2 - c_1|}{\sqrt{a^2 + b^2}}$$

Our lines in 2-D are:

$$w_1x_1 + w_2x_2 + b - 1 = 0 \text{ and } w_1x_1 + w_2x_2 + b + 1 = 0$$

$$\text{Distance} = \frac{|b - 1 - b - 1|}{\sqrt{w_1^2 + w_2^2}} = \frac{2}{\|\mathbf{w}\|}$$



Large Margin Linear Classifier

● denotes +1
○ denotes -1

Aim: Learn a large margin classifier

Mathematical Formulation:

$$\text{maximize } \frac{2}{\|\mathbf{w}\|}$$

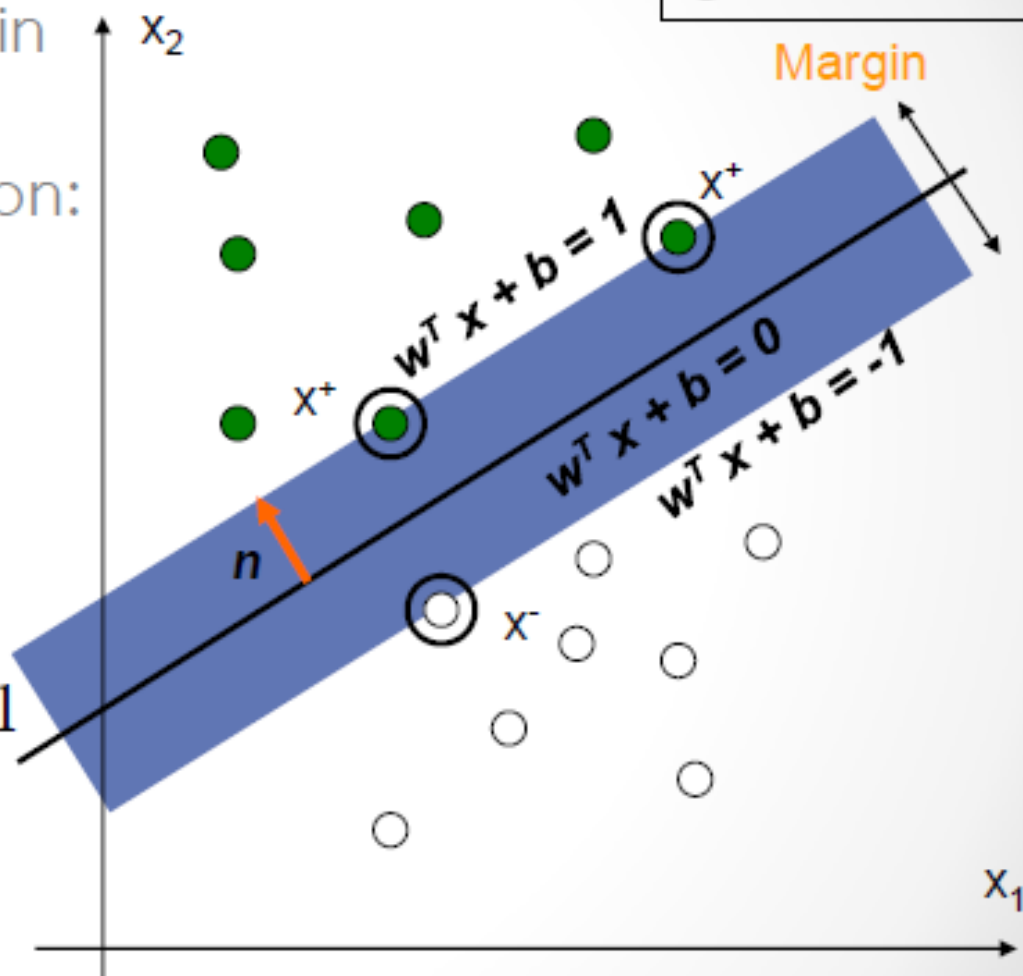
such that

$$\text{For } y_i = +1, \quad \mathbf{w}^T \mathbf{x}_i + b \geq 1$$

$$\text{For } y_i = -1, \quad \mathbf{w}^T \mathbf{x}_i + b \leq -1$$

such that

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$$



Large Margin Linear Classifier

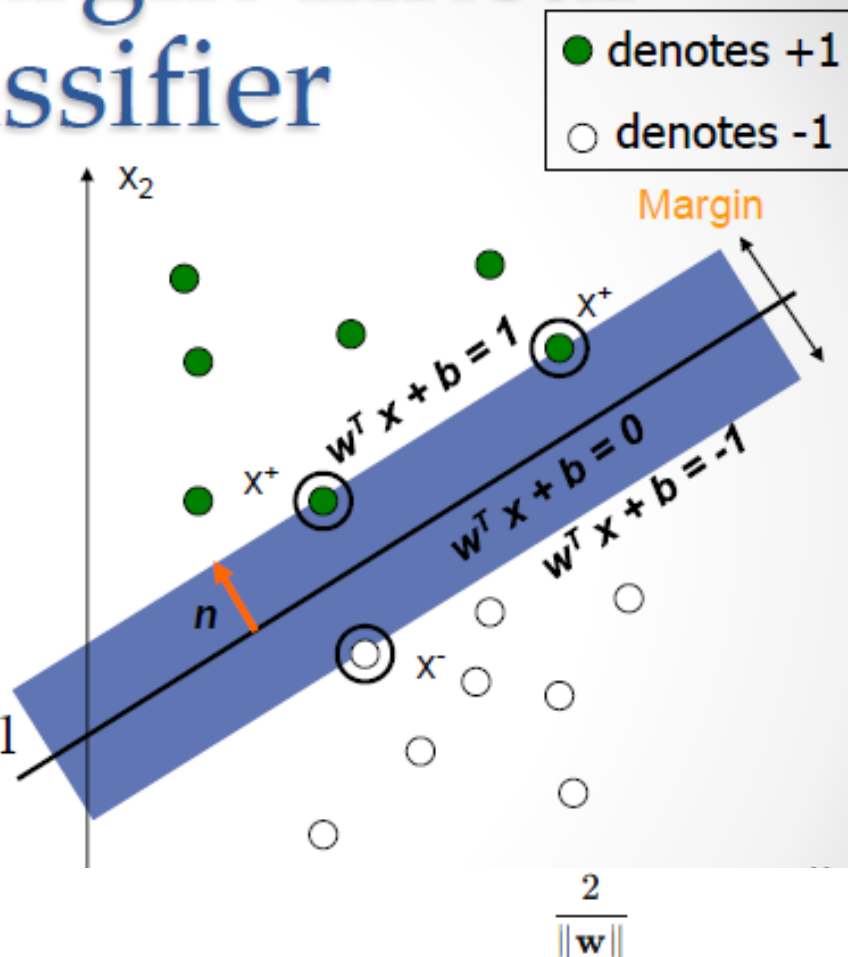
- Formulation:

$$\text{minimize } \frac{1}{2} \|\mathbf{w}\|^2$$

such that

$$\text{For } y_i = +1, \quad \mathbf{w}^T \mathbf{x}_i + b \geq 1$$

$$\text{For } y_i = -1, \quad \mathbf{w}^T \mathbf{x}_i + b \leq -1$$



This represents a fraction where 2 is a constant and $\|\mathbf{w}\|$ is the norm of the weight vector.

- As $\|\mathbf{w}\|$ decreases, the denominator becomes smaller, making the fraction larger.
- Conversely, as $\|\mathbf{w}\|$ increases, the fraction decreases.

Thus, maximizing $\frac{2}{\|\mathbf{w}\|}$ corresponds to minimizing $\|\mathbf{w}\|$.

Solving the Optimization Problem

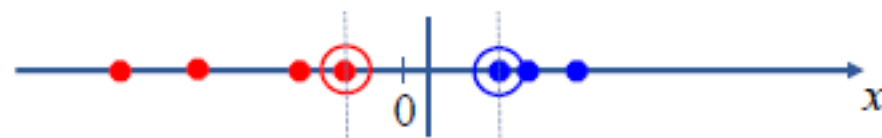
$$\begin{aligned} \text{minimize } L_p(\mathbf{w}, b, \alpha_i) &= \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i (y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1) \\ \text{s.t. } \alpha_i &\geq 0 \end{aligned}$$

$$\frac{\partial L_p}{\partial \mathbf{w}} = 0 \quad \Rightarrow \quad \mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$$

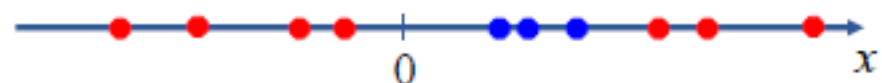
$$\frac{\partial L_p}{\partial b} = 0 \quad \Rightarrow \quad \sum_{i=1}^n \alpha_i y_i = 0$$

Non-linear SVMs

- Datasets that are linearly separable with noise work out great:

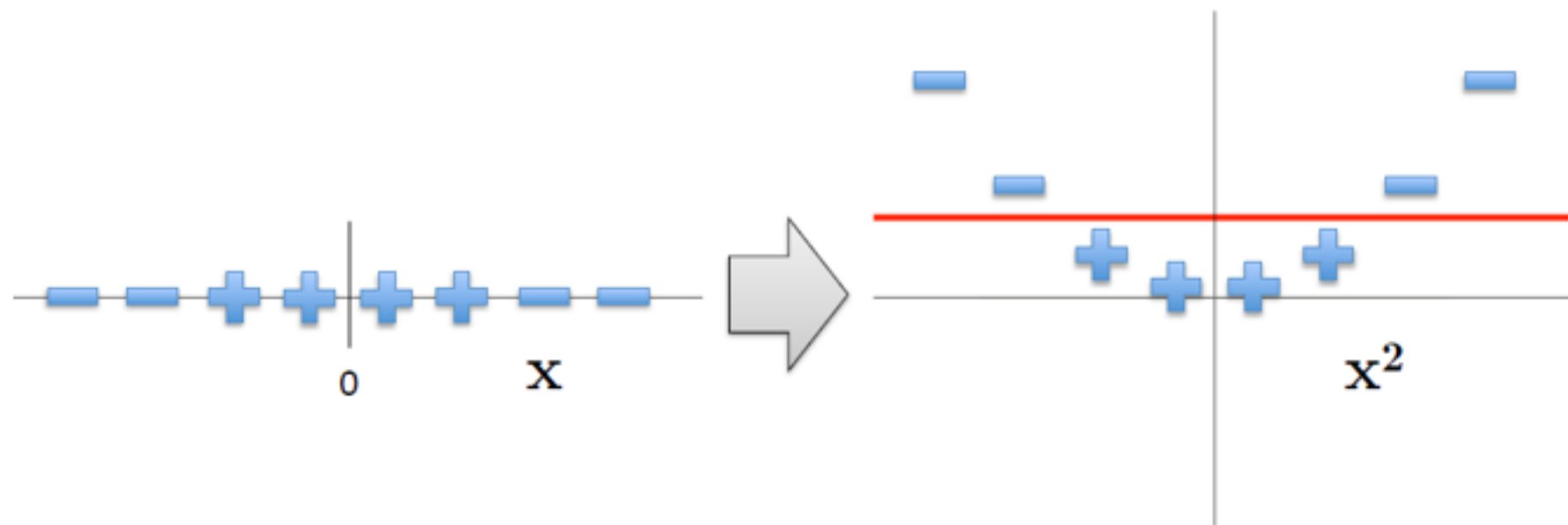


- But what are we going to do if the dataset is just too hard?



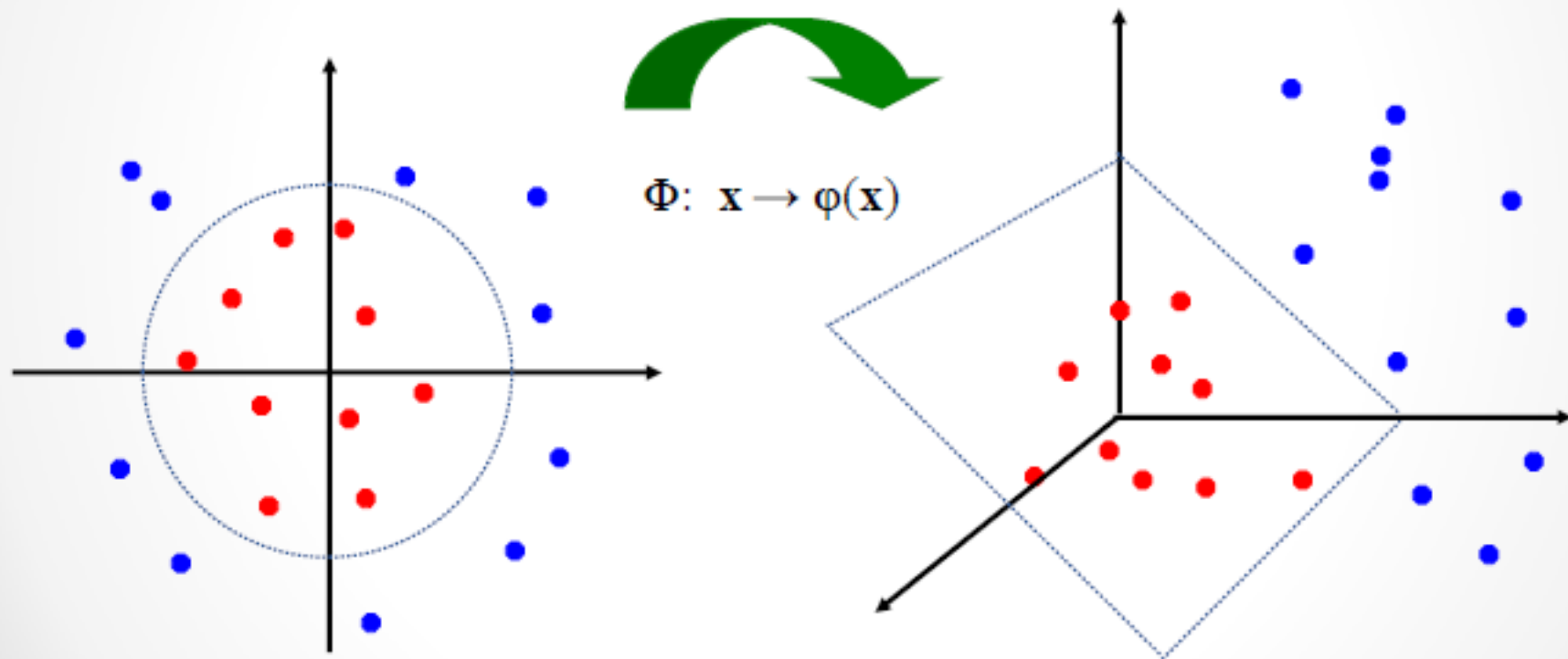
- Kernel Trick!!!
 - SVM = Linear SVM + Kernel Trick

When Linear Separators Fail



Non-linear SVMs: Feature Space

- General idea: the original input space can be mapped to some higher-dimensional feature space where the training set is separable:



Kernel Trick Motivation

- **Linear classifiers** are well understood, widely-used and efficient.
- How to use linear classifiers to build non-linear ones?

- **Kernels:**

- Map the problem from the input space to a new higher-dimensional space (called the feature space) by doing a non-linear transformation using a special function called the kernel.
- Then use a linear model in this new high-dimensional feature space. The linear model in the feature space corresponds to a non-linear model in the input space.

A Few Good Kernels...

- Linear Kernel $K(\mathbf{x}_i, \mathbf{x}_j) = \langle \mathbf{x}_i, \mathbf{x}_j \rangle$
- Polynomial kernel $K(\mathbf{x}_i, \mathbf{x}_j) = (\langle \mathbf{x}_i, \mathbf{x}_j \rangle + c)^d$
 - $c \geq 0$ trades off influence of lower order terms
- Gaussian kernel $K(\mathbf{x}_i, \mathbf{x}_j) = \exp \left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|_2^2}{2\sigma^2} \right)$
- Sigmoid kernel $K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\alpha \mathbf{x}_i^\top \mathbf{x}_j + c)$